TECHNISCHE UNIVERSITÄT BERLIN

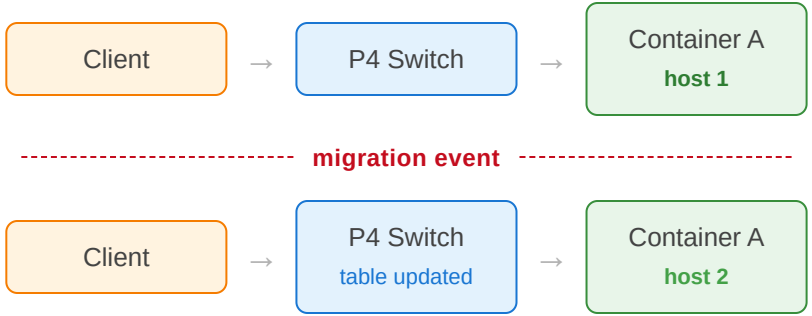
P4ContainerFlow

Live Container Migration with P4 Programmable Switches

Stanislav Kosorin | INET | Network Architectures - Master Project | March 10, 2026

Why Live Container Migration?

- VMs support live migration, but containers today are stopped and restarted.
- This kills all active TCP connections and causes visible downtime.
- Live migration would preserve running state and network connections.
- The client should never notice the container moved.



Same container, now on host 2. **TCP connection survives.**

Prior Work: GSoC 2024

Google Summer of Code 2024 · P4 Language Consortium

- CRIU checkpoints a running container and restores it on another host.
- A P4 switch redirects traffic to the new host transparently.
- Built a prototype on a software switch inside Kubernetes.
- TCP connections survived migration with zero drops.

Project Goals

> Prove that live container migration works on real switch hardware and measure what it costs.

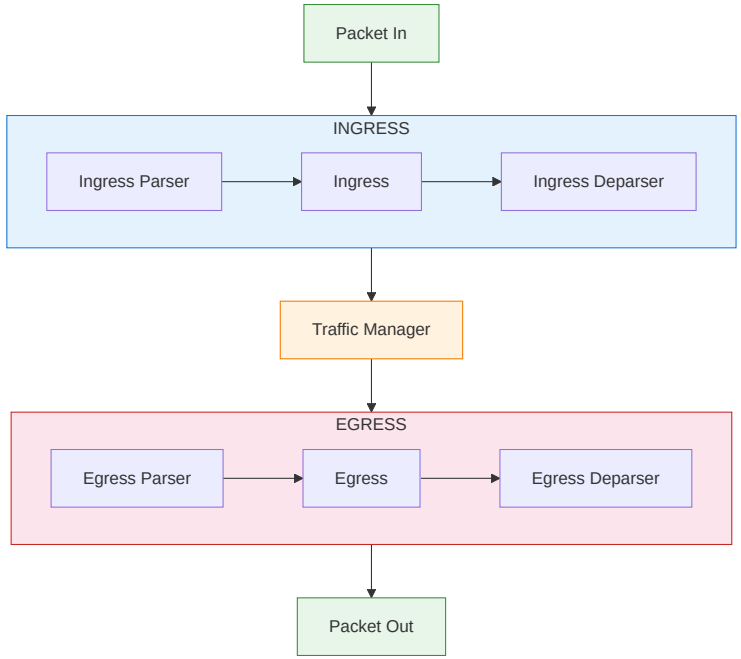
- Port the P4 load balancer to real Tofino switch hardware.
- Automate the full benchmark end-to-end.
- Measure migration timing, TCP survival, and recovery.
- Make everything reproducible from a VM to the physical switch.

Implementation

P4 architecture, development, and hardware

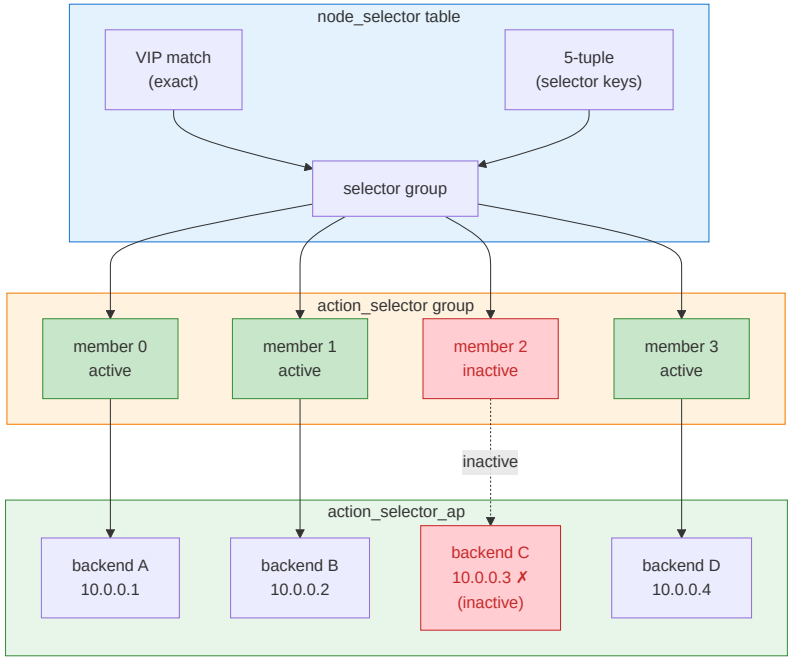
From V1Model to Tofino Native Architecture

- The original P4 program ran on BMv2, a software switch with no hardware limits.
- Tofino splits the pipeline into separate ingress and egress stages.
- The hardware compiler enforces strict resource constraints on each stage.
- Most of the P4 code had to be rewritten to fit the hardware model.



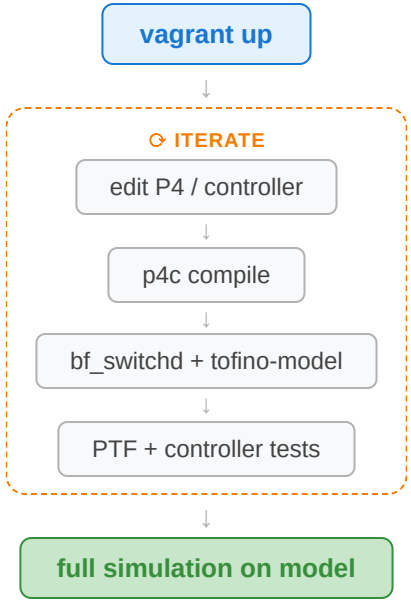
Rewriting the Load Balancer: ActionSelector

- The switch hashes each connection to a backend server.
- All packets in the same flow always reach the same server.
- Tofino requires a three-table structure: match table, action profile, and selector.
- BMv2 used a single table, the rewrite was the largest P4 change.



Reproducible Development with open-p4studio

- Intel open-sourced the Tofino SDE as open-p4studio in January 2025.
- A Vagrant VM builds the full toolchain in one command.
- The P4 program compiles and runs on a software model inside the VM.
- All development and testing happens without hardware access.



Getting to Hardware

- The compiled P4 binary is the same for model and hardware.
- Hardware requires proprietary drivers that are not in the open-source SDE.
- Setting up the switch involves manual steps not needed on the model.

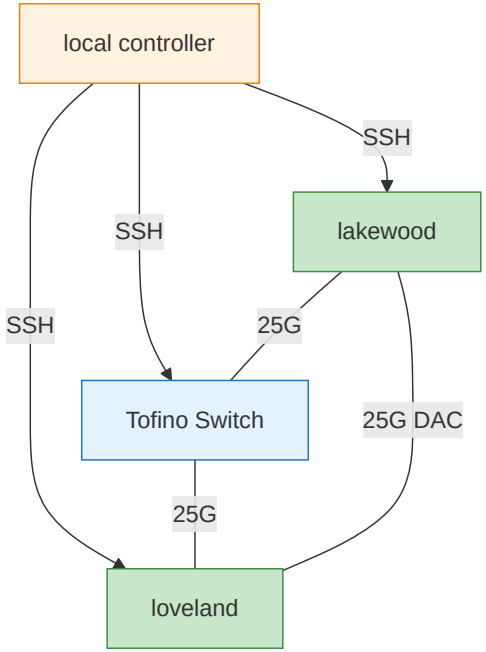
Component	Source	Type
P4 compiler + model	open-p4studio	open source
Serdes + port manager	Intel private archive	proprietary
Board Support Package	Intel private archive	proprietary
Kernel modules	built from source	manual

Experiment

Setup, workload, automation, and migration procedure

Testbed

- A Tofino 1 switch (Wedge100BF-32X) runs the P4 program and controller.
- Two servers act as container host and migration target.
- All container traffic is routed through the switch, even on the same host.
- A separate direct link between servers carries the migration data.



Workload

- A WebSocket server in the container sends timestamped frames at 30 fps.
- A load generator measures round-trip time across 4 concurrent connections.
- Before each migration, the server is signaled to finish in-flight data.

Component	Role	Host
Go server	WebSocket + metrics on :8081	container
Load generator	4 clients, RTT metrics on :9090	lakewood
Collector	scrapes both via SSH tunnel	control machine

Migration Procedure

- The server is signaled to stop sending so CRIU can capture a clean state.
- CRIU checkpoints the container and transfers it over the direct link.
- The container is restored with the same IP on the new host.
- The switch forwarding table is updated to point to the new port.

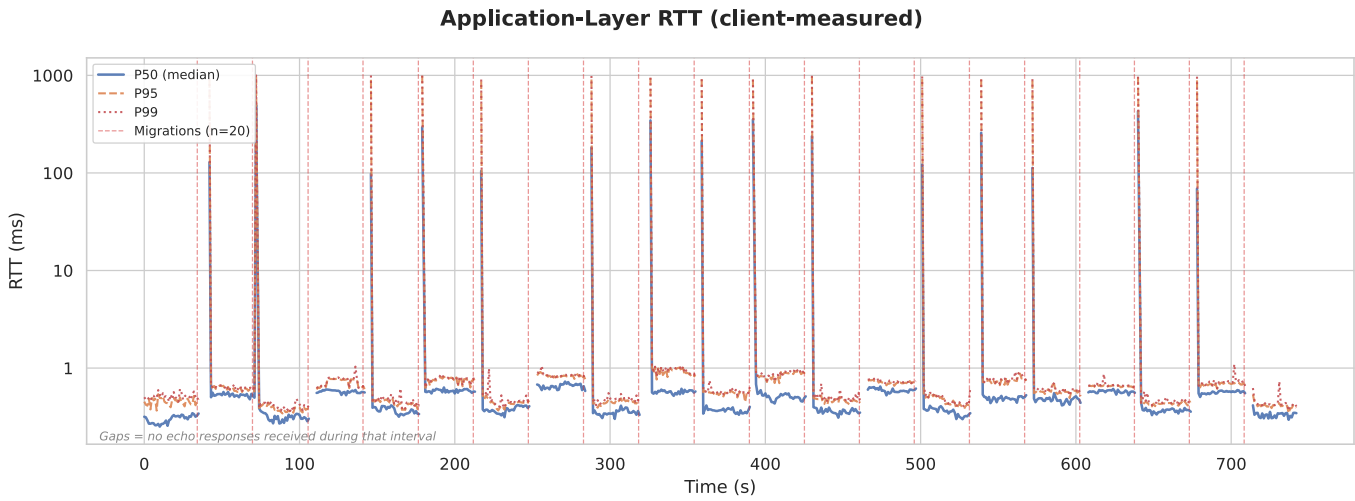
Phase	Notes
Checkpoint	quiesce, drain, CRIU dump
Pre-transfer	target prep, port setup
Transfer	socat over direct DAC link
Restore	largest contributor
Switch update	two gRPC calls

Results

It works. Zero dropped connections.

Results: Headline Numbers

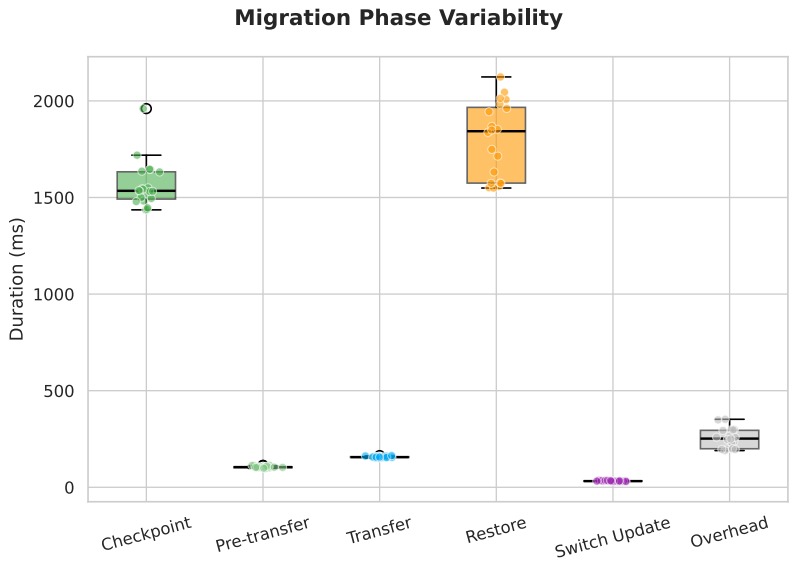
20	0	~4 s	~0.3 ms
Migrations	Dropped Connections	Median Downtime	Steady-State RTT (p50)



RTT over full run · dashed lines are migration events · log scale

Results: Phase Breakdown

- **Checkpoint:** ~1564 ms (quiesce, drain, CRIU dump)
- **Pre-transfer:** ~105 ms (target prep, port setup)
- **Transfer:** ~156 ms (direct-link `socat`)
- **Restore:** ~1797 ms (CRIU, macvlan, ARP)
- **Switch update:** ~32 ms (two gRPC calls)



Overhead: remaining `time_to_ready` not separately timestamped by the script.

Limitations

- Small synthetic workload, not large stateful applications.
- Dedicated transfer link, not a shared datacenter network.
- Same-IP migration only, not full VIP-based load balancing.
- Two servers, not a multi-node production cluster.

Future Work

Term	Next steps
near	Reduce restore time with CRIU lazy pages
near	Upstream build and driver fixes to the P4 community
mid	Multi-backend ActionSelector migration
long	Multi-node Kubernetes integration
long	Stateful workloads like Redis

Summary and Contributions

20	0	~4 s	~32 ms
Migrations	Dropped Connections	Median Downtime	Switch Update

- Proved live container migration works on real Tofino hardware at line rate.
- The switch updates in ~32 ms. The bottleneck is CRIU checkpoint and restore.
- Identified and solved CRIU restore failures caused by non-empty send queues.
- Navigated undocumented proprietary driver setup and published the full process.
- Entire stack is open source and reproducible from `vagrant up` to hardware.

Thank You!

Questions?

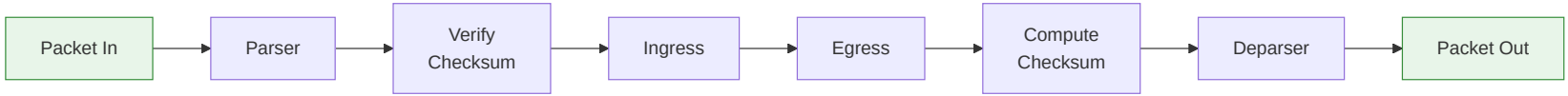
Appendix

References

- p4containerflow-tofino2 (this project): github.com
- p4containerflow (GSoC, BMv2): github.com
- GSoC 2024 project page: summerofcode.withgoogle.com
- open-p4studio: github.com
- Intel’s Tofino P4 Software is Now Open Source: p4.org
- P4-16 language specification: github.com
- CRIU TCP connection checkpoint/restore: criu.org
- podman container checkpoint: docs.podman.io
- checkpoint-restore/criu: github.com

V1Model: Single Linear Pipeline

- Six fixed blocks executed in a single linear sequence.
- One pipeline handles both ingress and egress.
- Software-simulated on BMv2 with no hardware constraints.
- Hashing, metadata, and checksums are simple and flexible.



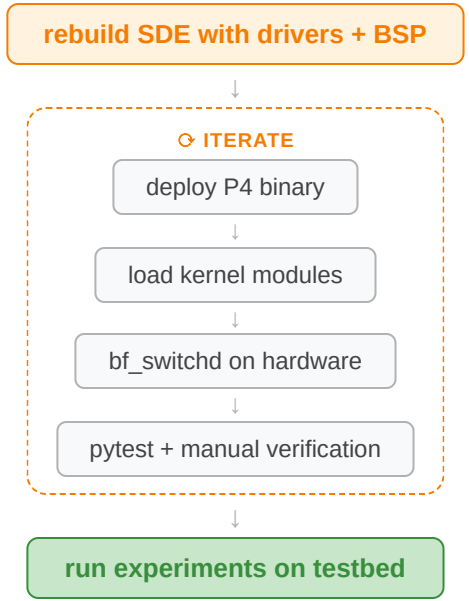
Testing on the Model First

- PTF sends real packets through virtual Ethernet pairs on the model.
- Controller integration tests drive the full stack via the HTTP API.
- All tests pass on the model before any hardware is touched.

Test suite	What it checks	Target
PTF data-plane	forwarding, SNAT, ARP	model + hw
PTF load balance	even backend distribution	model + hw
PTF migration	member update, flow continuity	model + hw
pytest controller	HTTP API, gRPC calls	model

Development on Hardware

- The SDE must be rebuilt with proprietary drivers and BSP.
- The compiled P4 binary from the model is deployed to the switch.
- Hardware tests use pytest and manual verification with real ports.
- The same iterate cycle applies until experiments succeed.



From TU Berlin to University of Vienna

- Initial development used a Tofino 2 at TU Berlin with shared access.
- We got access to a dedicated testbed at University of Vienna.
- Dedicated hardware made the full 20-migration benchmark possible.

TU Berlin

Tofino 2 · shared access

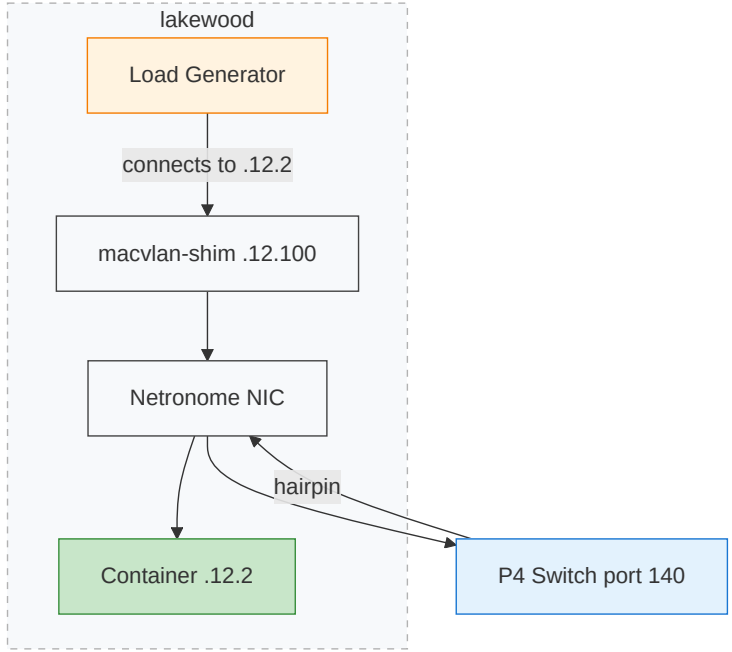
↓

University of Vienna

Tofino 1 · dedicated testbed

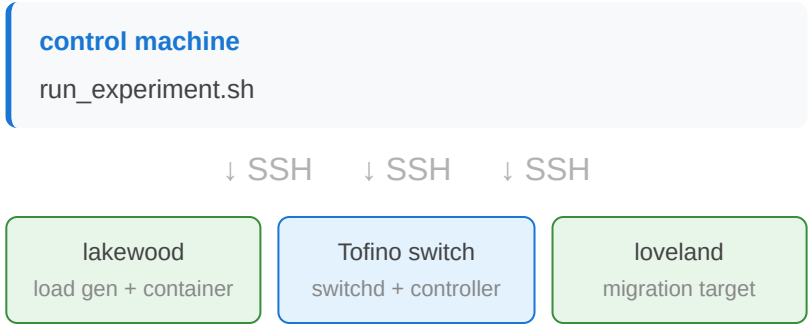
Experiment Setup: Networking

- VEPA mode forces all local container traffic through the switch (hairpin).
- A macvlan-shim on lakewood bypasses Linux macvlan host isolation.
- **Same-IP migration:** the container keeps `192.168.12.2` on both hosts.
- Only the switch forward table changes, not the container address.



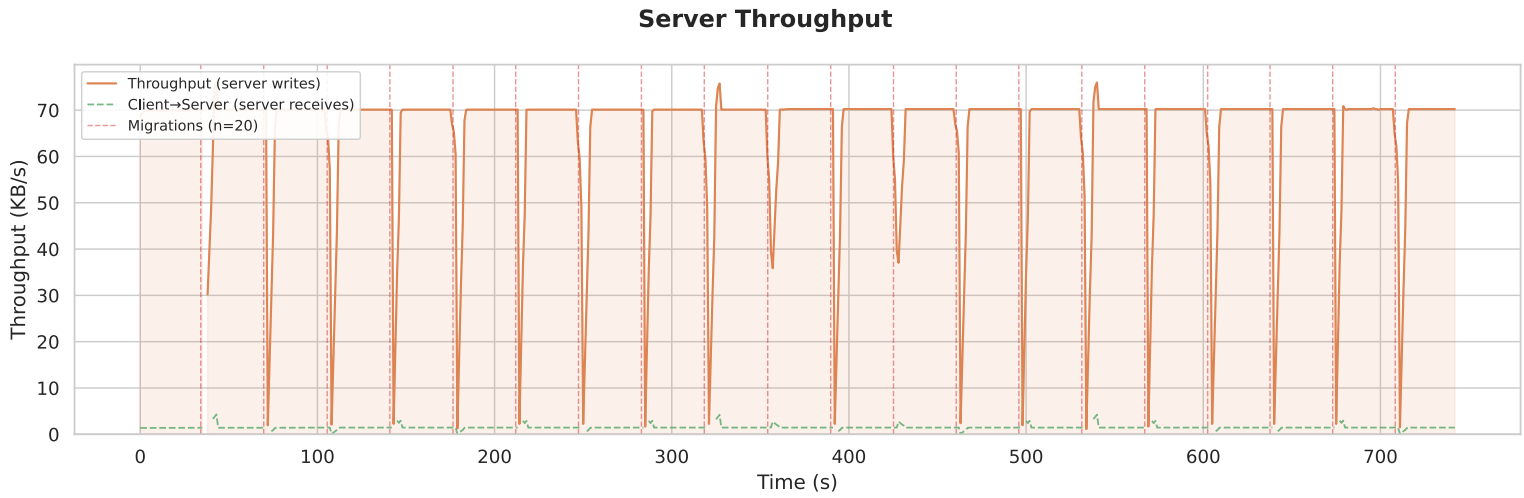
Experiment Automation

- A single script coordinates all nodes over SSH with agent forwarding.
- SSH multiplexing (`ControlMaster`) reuses connections to cut handshake overhead.
- The script syncs code, starts services, deploys containers, and runs migrations.
- Results go to a timestamped directory with automatic cleanup on failure.



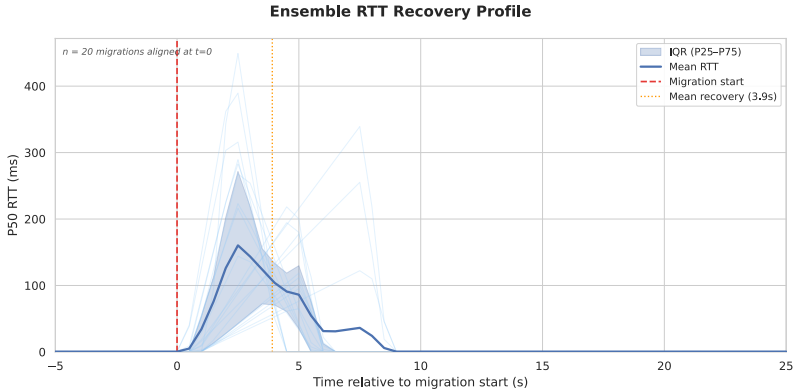
Results: Throughput

Throughput drops to zero during each freeze and recovers to steady state within seconds. Zero connection drops across all 20 migrations.

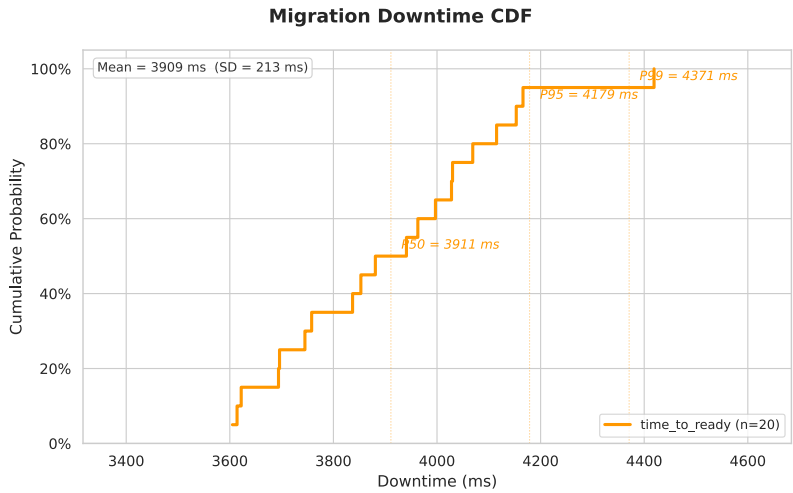


The TCP RTO Problem

- During the freeze, the client TCP stack backs off its retransmit timer exponentially.
- With sub-ms baseline RTT, the RTO grows to 15 to 30 s.
- **Fix:** set `rto_min 5ms` on the route and flush `ip tcp_metrics` after each migration.
- Recovery drops from 15 to 30 s down to 1 to 2 s.



20 migrations aligned at t=0 · IQR band

Results: Downtime Distribution

- Median downtime: 3881 ms (p50).
- 95th percentile: 4166 ms.
- All 20 migrations completed without a CRIU restore failure.
- Standard deviation: 212 ms across 20 runs.